ОТЧЁТ ПО ДОМАШНЕМУ ЗАДАНИЮ № 3

Анализ вакансий разработчиков мобильных платформ

Дисциплина: Инженерия данных

Вариант № 9 — «Программист, разработчик на мобильных платформах»

Выполнила: Нугаева Камилла

ИСУ: 434674

Группа: J3312

Репозиторий: github.com/kamii86/hh-skill-analytics

Октябрь 2026 года

1. Постановка задачи и подготовка данных

Цель работы — построить воспроизводимый аналитический процесс на базе объектного хранилища RustFS. Для вакансий своего варианта необходимо получить частоты навыков, пар и троек собственным потоковым MapReduce и независимым SQL-запросом DuckDB, проверить совпадение результатов и исследовать зарплаты, регионы и производительность.

1.1. Развёртывание RustFS и исходный файл

RustFS запущен локально как S3-совместимое хранилище. Исходный объект размещён в бакете hh-lake под ключом raw/Raw_Jobs.csv. Подключение проверено скриптом python -m src.check_s3; подтверждён размер файла 1 556 363 336 байт. Скриншот структуры бакета приведён в приложении А.

Характеристика	Результат
Записей в исходном файле	561 989
Отобрано уникальных вакансий варианта № 9	321
Вакансий с навыками	262
Устранено повторных ID	3
Вручную исключено нерелевантных вакансий	6

1.2. Отбор варианта № 9

Вакансии фильтруются по названиям должностей для Android, iOS и кроссплатформенной мобильной разработки. Исключаются тестировщики, аналитики, менеджеры и другие непрофильные роли. Шесть неоднозначных вакансий исключены после проверки описаний. Повторяющиеся ID исключаются; навыки нормализуются и дедублицируются внутри вакансии.

1.3. Архитектура

Источник	Обработка	Результат
RustFS / S3	Python MapReduce: чанки 1 000 / 5 000 / 10 000	CSV: навыки, пары, тройки
Тот же объект S3	DuckDB + httpfs: независимый SQL	CSV: навыки, пары, тройки
Результаты двух методов	Проверка ключей, частот и SHA-256	verification/
Выборка вакансий	Зарплаты, регионы, замеры времени и RAM	salary/, regions/, profiling/

Исходный CSV разделён символом «;»; поле описания содержит кавычки и может включать переносы строк. Для корректной потоковой обработки используется CSV-парсер, а не ручное разбиение файла по переводам строк.

2. Расчёт частот навыков

2.1. Собственная реализация MapReduce

Скрипт src/mapreduce_skills.py последовательно читает исходный объект из S3 чанками. На этапе Map формируются счётчики отдельных навыков и неупорядоченных сочетаний по два и три элемента. На этапе Reduce счётчики чанков суммируются. Это последовательная модель MapReduce, а не распределённый кластер.

Результат	Уникальных комбинаций	Всего вхождений
Навыки	383	1 622
Пары	3 674	6 039
Тройки	16 440	19 142

2.2. Почему количество сочетаний растёт

Если в одной вакансии присутствует k различных навыков, то формируется k одиночных навыков, C(k,2) = k(k−1)/2 пар и C(k,3) = k(k−1)(k−2)/6 троек. Например, для 10 навыков образуются 10 одиночных элементов, 45 пар и 120 троек. Этим объясняется рост суммарных вхождений от 1 622 одиночных навыков до 19 142 троек. При этом число уникальных комбинаций определяется повторяемостью сочетаний между вакансиями.

2.3. Частые навыки и сочетания

Навык	Вакансий	Навык	Вакансий
iOS	106	ООП	56
Git	104	Java	51
Swift	94	Android SDK	38
Android	89	Flutter	34
Kotlin	78	Objective-C	28

Наиболее частая пара	Частота	Наиболее частая тройка	Частота
iOS + Swift	70	Git + iOS + Swift	30
Git + iOS	52	—	—
Android + Kotlin	49	—	—

Полные выходные данные записаны в S3: mapreduce/chunk_1000/, mapreduce/chunk_5000/ и mapreduce/chunk_10000/.

3. Независимая реализация DuckDB и верификация

SQL-реализация в src/duckdb_skills.py читает исходный CSV непосредственно из S3 через расширение httpfs. Фильтр мобильных вакансий, исключения ID, нормализация навыков и удаление дубликатов отражены в SQL независимо от Python MapReduce. Для подсчёта пар и троек используются комбинации уникальных навыков одной вакансии.

Тип комбинаций	MapReduce	DuckDB	Расхождений
Навыки	383	383	0
Пары	3 674	3 674	0
Тройки	16 440	16 440	0
Итого	20 497	20 497	0

Полное совпадение ключей и частот: 20 497 из 20 497 комбинаций

3.1. Проверка размеров чанка

Скрипт src/verify_chunk_sizes.py сравнил SHA-256 трёх выходных CSV для размеров 1 000, 5 000 и 10 000 строк. Для skills.csv, pairs.csv и triples.csv контрольные суммы полностью совпали: результат не зависит от размера чанка.

3.2. Независимый контроль оптимизации

Выходные файлы исходного MapReduce и его оптимизированной версии при чанке 5 000 также сравнивались побайтово по SHA-256. Для навыков, пар и троек получены три совпадения.

3.3. Команды и доказательства

Действие	Команда
Собственный подсчёт	python -m src.mapreduce_skills --chunk-size 5000
Независимый SQL	python -m src.duckdb_skills
Сверка SQL и MapReduce	python -m src.verify_results
Сверка трёх чанков	python -m src.verify_chunk_sizes

Скриншот команды verify_results с тремя статусами PASS и нулевыми расхождениями приведён в приложении А. JSON-отчёт сохранён в RustFS: verification/report.json.

4. Анализ влияния сочетаний навыков на зарплату

Для анализа зарплат используются рублёвые предложения. При полном диапазоне «от X до Y RUR» вычисляется середина (X+Y)/2; таких вакансий 45. Для сравнения комбинации с одиночным навыком базовым выбирается навык с наибольшей общей частотой в отобранной выборке. Далее рассчитывается разность медиан: медиана вакансий с комбинацией минус медиана вакансий с базовым навыком. Для каждой строки требуется не меньше трёх зарплатных наблюдений.

4.1. Медиана середины полного интервала

Тип	Сочетание навыков	n	Базовый навык	Медиана базы, ₽	Медиана сочет., ₽	Δ, ₽
Пара ↑	ios + удаленная работа	3	ios	100 000	285 000	+185 000
Пара ↑	android sdk + java	3	java	100 000	275 000	+175 000
Пара ↑	android + android sdk	5	android	120 000	245 000	+125 000
Пара ↓	java + kotlin	4	kotlin	140 000	97 500	−42 500
Пара ↓	android + java	7	android	120 000	100 000	−20 000
Пара ↓	dart + flutter	5	flutter	85 000	70 000	−15 000
Тройка ↑	android + android sdk + java	3	android	120 000	275 000	+155 000
Тройка ↑	android + android sdk + kotlin	3	android	120 000	245 000	+125 000
Тройка ↑	git + ios + swift	6	ios	100 000	102 500	+2 500
Тройка ↓	android + java + kotlin	3	android	120 000	100 000	−20 000
Тройка ↓	android + git + ios	3	ios	100 000	90 000	−10 000
Тройка ↓	git + ios + ооп	3	ios	100 000	90 000	−10 000

Пример: у сочетания Android SDK + Java медиана равна 275 000 ₽ (n=3), а у наиболее частого из этих навыков, Java, — 100 000 ₽. Разница +175 000 ₽ не доказывает причинного эффекта навыка, поскольку уровни должностей и вакансий могут различаться.

В исходном поле key_skills встречается «удаленная работа». Этот тег участвует в формальном подсчёте, но описывает формат занятости, а не технический навык; высокую разницу для iOS + «удаленная работа» нельзя интерпретировать как эффект освоения новой технологии.

5. Зарплатный анализ: нижняя граница

Чтобы не ограничиваться вакансиями с полным диапазоном, отдельно изучена указанная нижняя граница зарплаты в рублях (112 вакансий). Это другой показатель: его медиану нельзя напрямую сравнивать с медианой середины полного интервала. Ниже приведены наиболее выраженные положительные и отрицательные различия для пар и троек.

Тип	Сочетание навыков	n	Базовый навык	Медиана базы, ₽	Медиана сочет., ₽	Δ, ₽
Пара ↑	kotlin + mvvm	3	kotlin	90 000	200 000	+110 000
Пара ↑	android + mvvm	3	android	95 000	200 000	+105 000
Пара ↑	git + swiftui	3	git	85 000	180 000	+95 000
Пара ↓	android + firebase	3	android	95 000	50 000	−45 000
Пара ↓	android sdk + android studio	3	android sdk	140 000	100 000	−40 000
Пара ↓	dart + ios	4	ios	100 000	65 000	−35 000
Тройка ↑	android + kotlin + mvvm	3	android	95 000	200 000	+105 000
Тройка ↑	git + ios + rest api	3	ios	100 000	200 000	+100 000
Тройка ↑	git + swift + удаленная работа	3	git	85 000	180 000	+95 000
Тройка ↓	dart + git + ios	3	ios	100 000	50 000	−50 000
Тройка ↓	flutter + git + ios	3	ios	100 000	50 000	−50 000
Тройка ↓	android + git + ios	4	ios	100 000	55 000	−45 000

5.1. Интерпретация

Для сочетания Kotlin + MVVM нижняя медиана зарплаты равна 200 000 ₽, для базового Kotlin — 90 000 ₽ (разница +110 000 ₽; n=3). Для Android + Firebase разница обратная: 50 000 ₽ против 95 000 ₽ (−45 000 ₽; n=3). Снижение не означает, что Firebase ухудшает оплату: результат может объясняться долей начинающих специалистов, условиями работы и малой выборкой.

В результате расчёта при пороге n≥3 получено: для середины интервала — 9 положительных и 9 отрицательных пар, 3 положительные и 3 отрицательные тройки; для нижней границы — 41/25 пар и 39/24 троек соответственно. Полные таблицы доступны в префиксах salary/midpoint/ и salary/lower/ RustFS.

6. Сравнительный анализ трёх регионов

Выбраны Москва, Санкт-Петербург и Казань. В первой таблице сравниваются объём выборки и две отдельно рассчитанные зарплатные метрики. Сведения ограничены вакансиями, для которых определён соответствующий регион.

Таблица 1. Количество вакансий и зарплаты

Показатель	Москва	Санкт-Петербург	Казань
Всего вакансий	49	22	5
С заполненными навыками	45	19	3
Полный интервал: наблюдений	6	2	2
Медиана середины интервала, ₽	92 500	170 000	145 000
Нижняя граница: наблюдений	17	7	4
Медиана нижней границы, ₽	70 000	140 000	85 000

Таблица 2. Востребованность навыков и сочетаний

Регион	Частые навыки	Лидирующая пара	Лидирующая тройка
Москва	iOS — 22 (44,9 %) Swift — 21 (42,9 %) Git — 17 (34,7 %)	iOS + Swift — 18	Git + iOS + Swift — 8
Санкт-Петербург	Android, Git, iOS и Swift — по 6 (27,3 %)	iOS + Swift — 6	iOS + Objective-C + Swift — 4
Казань	Git — 3 (60 %) Objective-C, Realm, Swift — по 2 (40 %)	Git + Objective-C — 2*	Git + Objective-C + Realm — 2*

* Для Казани несколько пар и троек делят первое место; указана одна из комбинаций с максимальной частотой. Доли рассчитывались от всех вакансий соответствующего города, включая вакансии без навыков.

Наблюдения по регионам

В Москве преобладают сочетания iOS и Swift; они встречаются совместно в 18 из 49 вакансий. В Петербурге одинаково часто упоминаются четыре технологии (по 6 вакансий), а iOS + Swift остаётся лидирующей парой. В Казани найдено только 5 вакансий: доли и медианы существенно чувствительны к единичным наблюдениям. Особенно осторожно следует сравнивать медианы полного зарплатного интервала Петербурга и Казани, рассчитанные всего по двум вакансиям.

Полные результаты по регионам: regions/region_summary.csv, region_skills.csv, region_pairs.csv и region_triples.csv.

7. Профилирование времени и памяти

Проведено по три запуска для MapReduce с чанками 1 000, 5 000 и 10 000 строк, а также DuckDB. Измерено время выполнения каждого процесса и его пиковая резидентная память (RSS). В таблице — средние значения трёх запусков, а не наибольший пик за всю серию.

Вариант	Запусков	Время, с (средн.)	RSS, МиБ (средн. пик)
MapReduce / 1 000	3	21,744	87,34
MapReduce / 5 000	3	13,733	125,34
MapReduce / 10 000	3	15,270	168,67
DuckDB	3	18,169	247,62

Рисунок 1 — Среднее время выполнения (три запуска)

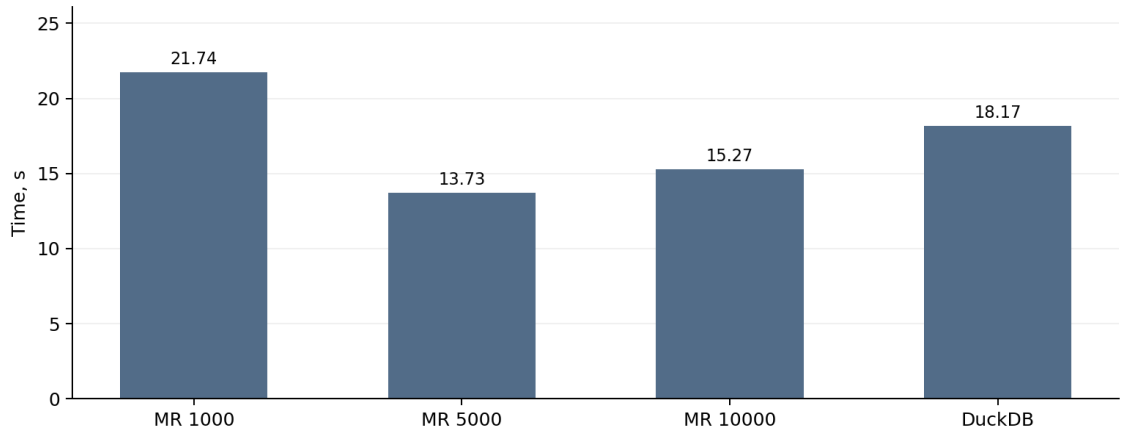
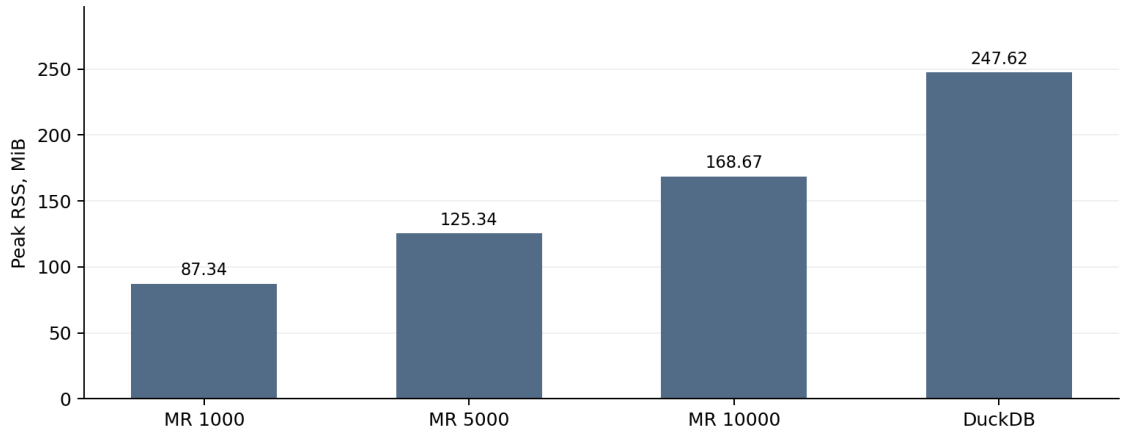


Рисунок 2 — Средняя пиковая RSS (три запуска)



Минимальное среднее время показал MapReduce при чанке 5 000 строк (13,733 с); наименьшее потребление памяти — чанк 1 000 (87,34 МиБ). DuckDB в этих условиях занял в среднем 18,169 с и 247,62 МиБ. Это результаты конкретного компьютера, а не общая гарантия производительности.

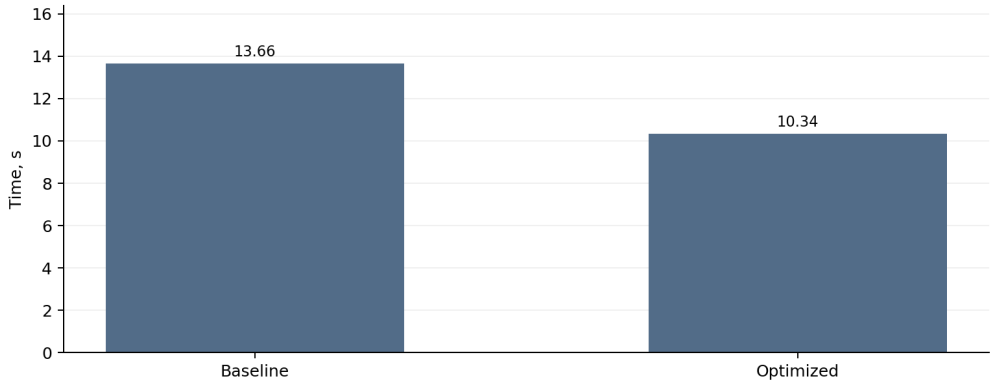
8. Оптимизация и проверка восстановления

8.1. Сравнение исходной и оптимизированной версии

В оптимизированном MapReduce промежуточные словари-счётчики для каждого чанка заменены прямым обновлением общих Counter. Для обеих версий выполнено по три запуска при чанке 5 000 строк. Порядок запусков чередовался, чтобы уменьшить влияние внешних факторов.

Показатель	Исходная версия	Оптимизированная
Среднее время, с	13,659	10,344
Средняя пиковая RSS, МиБ	125,68	123,74
Изменение	—	−24,27 % времени; −1,54 % RSS

Рисунок 3 — Эффект оптимизации времени выполнения



Контрольные суммы выходных CSV исходной и оптимизированной версий совпадают для навыков, пар и троек. Таким образом, ускорение не связано с изменением результата.

8.2. Тест отказа маппера

При моделировании ошибки маппера процесс завершился с кодом 1; ранее опубликованные результаты в RustFS не изменились. Полный повторный запуск без ошибки завершился с кодом 0, контрольные суммы skills.csv, pairs.csv и triples.csv совпали. Проверено восстановление повторным вычислением, не автоматический перезапуск отдельного чанка.

9. Воспроизводимость и выводы

9.1. Программные файлы и расположение результатов

Состав работ	Основные файлы или объекты
Загрузка и подключение	src/config.py; src/check_s3.py; s3://hh-lake/raw/Raw_Jobs.csv
MapReduce и фильтрация	src/mapreduce_skills.py; src/variant_filter.py; src/vacancy_quality.py
SQL и проверка	src/duckdb_skills.py; src/verify_results.py; src/verify_chunk_sizes.py
Зарплаты и регионы	src/salary_analysis.py; src/regional_analysis.py
Профилирование и бонус	src/profile_runs.py; src/benchmark_optimization.py; src/mapreduce_optimized.py
Отказ и восстановление	src/test_mapper_failure.py
Результаты	RustFS: mapreduce/, duckdb/, salary/, regions/, profiling/, optimized/, optimization/, verification/

9.2. Основные команды проверки

Действие	Команда
Доступ к RustFS	python -m src.check_s3
MapReduce / 5 000	python -m src.mapreduce_skills --chunk-size 5000
Независимый SQL	python -m src.duckdb_skills
Сверка результатов	python -m src.verify_results
Зарплаты и регионы	python -m src.salary_analysis ; python -m src.regional_analysis
Профилирование	python -m src.profile_runs
Оптимизация	python -m src.benchmark_optimization
Тест отказа	python -m src.test_mapper_failure

9.3. Итог

Выполнен расчёт частот навыков, пар и троек по вакансиям разработчиков мобильных платформ с прямым чтением из S3. Независимые алгоритмы MapReduce и DuckDB согласованы по всем 20 497 комбинациям, результаты трёх размеров чанков идентичны. Исследованы зарплаты и три региона; проведено профилирование и продемонстрировано сокращение среднего времени на 24,27 % после оптимизации. Тест сбоя подтвердил сохранность результатов в хранилище и корректное восстановление полным перезапуском.

Ограничения: исторический датасет, небольшие зарплатные и региональные подвыборки, атрибуты занятости в поле навыков, последовательная (не кластерная) реализация MapReduce и зависимость временных замеров от среды запуска.

Репозиторий: https://github.com/kamii86/hh-skill-analytics

Приложение A. Подтверждение RustFS и проверки

Рисунок A.1 — Структура бакета hh-lake в консоли RustFS

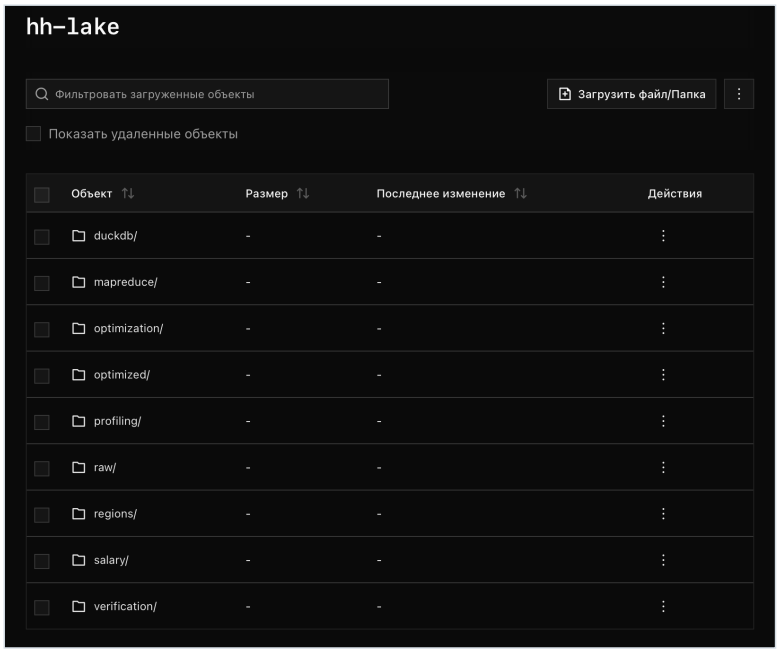


Рисунок A.2 — Проверка MapReduce против DuckDB: все частоты совпали

```
(.venv) (base) kamillanugaeva@Noutbuk-Kamilla hw3 % cd ~/rustfs-hw/hw3
python -m src.verify_results
============================================================
MAPREDUCE VS DUCKDB — FULL VERIFICATION
============================================================

SKILLS
--------------------------------------------------
MapReduce combinations: 383
DuckDB combinations: 383
Checked combinations: 383
Differences: 0
RESULT: PASS

PAIRS
--------------------------------------------------
MapReduce combinations: 3674
DuckDB combinations: 3674
Checked combinations: 3674
Differences: 0
RESULT: PASS

TRIPLES
--------------------------------------------------
MapReduce combinations: 16440
DuckDB combinations: 16440
Checked combinations: 16440
Differences: 0
RESULT: PASS

============================================================
FINAL RESULT: ALL RESULTS MATCH!
Report: s3://hh-lake/verification/report.json
============================================================
```

Приложение Б. Результаты профилирования

Рисунок Б.1 — CSV со средними временами и пиковой RSS по трём прогонам

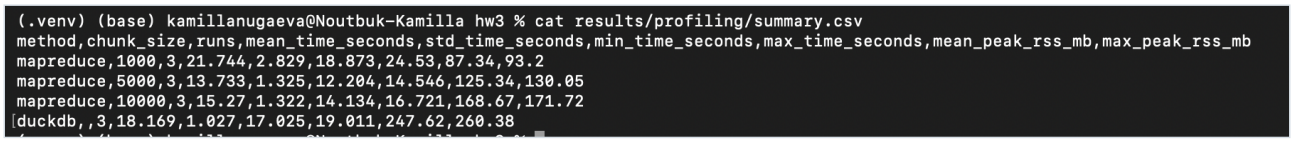
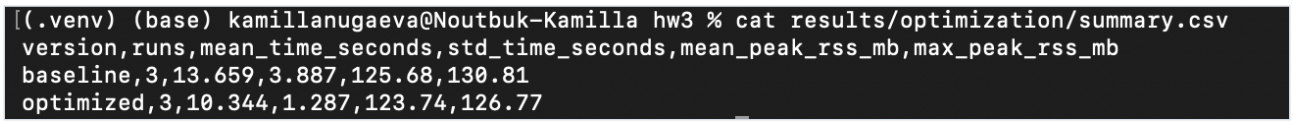


Рисунок Б.2 — CSV сравнения baseline и optimized



В результате повторных измерений получено: MapReduce / 5 000 — 13,733 с, 125,34 МиБ; DuckDB — 18,169 с, 247,62 МиБ. Оптимизация уменьшила среднее время с 13,659 до 10,344 с и среднюю пиковую RSS с 125,68 до 123,74 МиБ.

Приложение B. Отказ маппера и восстановление

Рисунок B.1 — Сбой, сохранность S3-результатов и повторный успешный запуск

```
(.venv) (base) kamillanugaeva@Noutbuk-Kamilla hw3 % python -m src.test_mapper_failure
======================================================================
ТЕСТ ОТКАЗА МАППЕРА
======================================================================

1. Запоминаем контрольные суммы результатов...

2. Запускаем программу с неисправным маппером...
Код завершения: 1
PASS: ошибка маппера вызвала аварийное завершение

3. Проверяем сохранность результатов...
PASS: результаты в RustFS не изменились

4. Повторяем вычисление без ошибки...
Код завершения: 0
PASS: повторный запуск успешен

5. Сравниваем восстановленные результаты...
SKILLS: PASS
PAIRS: PASS
TRIPLES: PASS

======================================================================
FINAL RESULT: FAILURE AND RECOVERY TEST PASSED!
======================================================================

Логи:
results/failure_test/mapper_failure.log
results/failure_test/recovery.log
```

Искусственный отказ приводит к ненулевому коду завершения. После повторного запуска контрольные суммы трёх выходных CSV совпадают с исходными.